



universidad de buenos aires - exactas

departamento de física

laboratorios de enseñanza

Controlador de temperatura para los laboratorios de enseñanza de física

Compendio de I+D, revisiones técnicas y
manual de uso

Marcos Damián Perez

Facultad de Ciencias Exactas y Naturales

Departamento de Física

Licenciatura en Ciencias Físicas

Laboratorios de enseñanza

Buenos Aires, junio 2026

Controlador de temperatura para los laboratorios de enseñanza de física

Compendio de I+D, revisiones técnicas y manual de uso

Marcos Damián Perez

Supervisor: Cobelli, Pablo

Profesor , Departamento de física

Coordinador: Alejandro César Greco

Coordinador, Departamento de física

Pañoleros: Alejandro

Damián

Federico

Juanita

Mateo

Magali

Agustina

Yamila

mas...

Facultad de Ciencias Exactas y Naturales

Departamento de Física

Licenciatura en Ciencias Físicas

Laboratorios de enseñanza

Cuaderno

Buenos Aires, junio 2026

Controlador de temperatura para los laboratorios de enseñanza de física

Copyright © 2026 - Marcos Damián Perez, Facultad de Ciencias Exactas y Naturales.

Este trabajo ... descargo correspondiente.

Agradecimientos

Guía de Escritura

Agradecimientos a quien corresponda.

1

Resumen

Documentacion del controlador de temperatura desarrollado para laboratorio 4 de la carrera de licenciatura en fisica de la universidad de buenos aires. El documento incluye una descripción detallada del diseño, desarrollo y pruebas del controlador, así como un análisis de su desempeño y posibles mejoras futuras. El objetivo principal es proporcionar una herramienta para la medición y control de la temperatura en experimentos de laboratorio, con un enfoque en la precisión, estabilidad y facilidad de uso. El controlador de temperatura desarrollado es capaz de mantener una temperatura constante con una alta precisión, lo que lo hace ideal para una amplia gama de aplicaciones en el laboratorio.

Índice general

1. Resumen	III
<i>Índice de figuras</i>	VIII
<i>Índice de cuadros</i>	X
<i>Glosario</i>	XII
<i>Siglas</i>	XIV
<i>Símbolos</i>	XVI
I Estado Actual del Sistema	1
2. Especificaciones	2
2.1. Actualizacion, 2 de junio de 2026	2
2.2. historial de cambios:	2
3. Arquitectura General del Amplificador	3
3.1. El controlador de temperatura	3
3.1.1. Estado del arte	3
3.1.2. Etapa de salida	4
3.1.3. Medición de corriente y tensión	4
3.1.4. Sistemas de protección	4
3.2. El panel de frontal	4
3.2.1. Controles	4
3.2.2. Indicadores	4
3.2.3. Display	4
3.3. Interfaz de control remoto y programación	4
3.3.1. Interfaces de comunicación	4
3.3.2. Protocolos de comunicación	4
3.3.3. Comandos SCPI	4
4. Controlador-de-Temperatura	5
5. Control y Medición	6

6. Protecciones	7
7. Alimentación	8
8. Interfaz de Usuario	9
9. Diseño Térmico	10
10. Diseño Mecánico	11
11. Integración Electromecánica	12
12. Implementacion Electronica	13
12.1. Esquematicos	13
12.2. PCBs	13
13. Implementacion Mecanica	14
14. Implementación de Firmware	15
14.1. dependencias	15
15. Validación Eléctrica	16
16. Validación Térmica	17
17. Validación del Sistema	18
18. Manual de Operación	19
19. Mantenimiento	20
 II Gestión de Versiones del Producto	 21
20. Historial de Versiones	22
 III Desarrollo de Firmware	 23
21. Desarrollo de firmware	24
21.1. Resumen	24
22. Sistema de Comunicación	25
22.1. Arquitectura General	25
22.1.1. Modelo de Capas y Responsabilidades	25
22.1.2. Propiedad de Recursos	27
22.1.3. Conceptos Fundamentales	30
22.1.4. Flujo de Comunicación	32

22.2. Capa de Driver	34
22.2.1. Interfaz Abstracta <code>comm_iface_t</code>	34
22.2.2. Estados de la Interfaz	36
22.2.3. Eventos de Comunicación y Error	36
22.2.4. APIs de Ciclo de Vida	36
22.2.5. APIs de Acceso a Datos	36
22.2.6. Protección de Acceso Concurrente	36
22.2.7. Estrategias de Sincronización y Gestión de Buffers	36
22.3. Capa de Middleware (Communication Task)	36
22.3.1. Modelo de Ejecución y Planificación	36
22.3.2. Registro de Eventos y Captura Atómica (<i>Snapshot</i>)	36
22.3.3. Orden de Precedencia y Flujo de Control	36
22.3.4. Máquinas de Estado de Transporte	36
22.3.5. API de Servicios hacia la Capa de Aplicación	36
22.3.6. Adaptación a Protocolos de Instrumentación	36
22.3.7. Selector de Medio de Transporte	36
22.3.8. Propiedad de Recursos	37
22.4. Interfaz de la Capa Física (Driver)	37
22.4.1. Responsabilidades por Componente	39
22.5. Conceptos Fundamentales	39
22.6. Arquitectura de las Máquinas de Estados (Modelo Mealy)	40
22.6.1. Máquina de Gestión de Interfaz	41
22.6.2. Máquina de Recepción (RX)	41
22.6.3. Máquina de Transmisión (TX)	42
22.6.4. Máquina de Procesamiento SCPI	42
22.7. Modelo de Interacción	42
22.8. Selector de Medio de Transporte	43
22.9. Gestión Interna de Buffers en el Driver	44
22.10 Mecanismo de Despacho de Eventos y Priorización de Ejecución	45
22.10.1 Patrón de Captura Instantánea (<i>Snapshot</i>) Atómica	45
22.10.2 Orden de Precedencia y Flujo de Control	46
22.10.3 Adaptación al Protocolo SCPI	46
22.11 Entorno de Ejecución y Planificación	47
22.12 Integración del Parser SCPI (<code>libscpi</code>)	48
23. Motor SCPI	49
23.1. Integración del parser <code>libscpi</code>	49
23.2. Flujo de recepción y análisis de comandos	49
23.3. Generación y transmisión de respuestas	49

Anexos

Índice de figuras

22.1. Modelo de capas del sistema de comunicación. Las flechas indican las dependencias funcionales y el sentido del flujo de datos.	25
22.2. Estructura <code>comm_iface_t</code> y su interacción con la <i>Communication Task</i> y el driver concreto.	35
22.3. Sistema de comunicación con FSMs tipo Mealy.	43

Índice de cuadros

22.1. Propiedad de recursos en el sistema de comunicación.	27
--	----

Parte I

Estado Actual del Sistema

2

Especificaciones

2.1 Actualizacion, 2 de junio de 2026

Especificaciones del sistema en desarrollo. Estas especificaciones pueden ser modificadas a lo largo del proyecto, pero es importante tener una base clara desde el principio para guiar el diseño y desarrollo del controlador de temperatura.

- Seguridad: El diseño debe cumplir con las normativas de seguridad del laboratorio, evitando riesgos para los estudiantes y el equipo.
- Corriente máxima de salida: 10 A continuos (DC).
- Tensión máxima de salida: 24 V.
- Ancho de banda: DC a 10 Hz.
- Precisión en DC: offset menor a 5 mV (si puede ser menos, mejor).
- Ruido de salida menor a 10 mV
- Sistemas de protección: El diseño debe incluir protecciones contra sobrecorriente, sobretensión, cortocircuitos y sobrecalentamiento para garantizar la seguridad y la durabilidad del equipo.
- Interfaz de control: El controlador debe ser controlable a través de una interfaz digital (por ejemplo, USB, Ethernet o RS-232) para permitir su integración con sistemas de adquisición de datos y controladores externos.
- El equipo debe soportar comandos SCPI para facilitar su integración con software de control y adquisición de datos comúnmente utilizado en laboratorios.
- Interfaz de usuario: El Controlador debe contar con una interfaz de usuario intuitiva para facilitar su operación por parte de los estudiantes y el personal del laboratorio, incluyendo indicadores de estado y controles de ajuste de la salida.

2.2 historial de cambios:

- **2 de junio del 2026:** Se establecieron las especificaciones del sistema en desarrollo.

3

Arquitectura General del Amplificador

En esta sección se presenta una descripción general de la arquitectura del amplificador lineal de potencia, destacando los componentes principales y su función dentro del sistema. Se abordarán aspectos como la etapa de entrada, la etapa de amplificación, la etapa de salida y los sistemas de protección y control.

3.1 El controlador de temperatura

Actualmente para no hay en el alboratorio un controlador de temperatura y solo se deja evolucionar la temperatura midiendo en los trancitorios y estacionarios. El objetivo de este proyecto es desarrollar un controlador de temperatura que permita mantener una temperatura constante con alta precisión, lo que es fundamental para una amplia gama de experimentos en el laboratorio, no solo para tener la bariable controlada sino tambien para acelerar los largo transitorios de termalización de los experimentos. El controlador de temperatura desarrollado se integrará con el sistema de adquisición de datos del laboratorio, permitiendo un control preciso y una fácil monitorización de la temperatura durante los experimentos.

3.1.1 Estado del arte

- Etapa de salida: Puentes H con transistores MOSFET de potencia en configuración push-pull para manejar las altas corrientes y tensiones requeridas, con un diseño que minimice las pérdidas de conmutación y maximice la eficiencia.
- Control de la etapa de salida: modulación por ancho de pulso (PWM) para controlar la potencia entregada a la carga, con un controlador de retroalimentación para mantener la temperatura constante.
- Filtro de salida: Un filtro LC para suavizar la señal de salida y reducir el ruido, asegurando una entrega de potencia estable y precisa a la carga.

- Control y medición: Un microcontrolador o FPGA para implementar el control del amplificador, la adquisición de datos de los sensores de corriente y tensión, y la comunicación con la interfaz de usuario.
- Sistemas de protección: Circuitos de protección contra sobrecorriente, sobretensión, cortocircuitos y sobrecalentamiento para garantizar la seguridad y la durabilidad del equipo.
- Medición de corriente y tensión: sensores de corriente y tensión de alta precisión para monitorear la salida del amplificador y proporcionar retroalimentación al controlador.
- Interfaz de usuario: Pantalla LCD o LED para mostrar el estado del amplificador, controles de ajuste para configurar la salida y botones para encender/apagar el dispositivo.

3.1.2 Etapa de salida

3.1.3 Medición de corriente y tensión

3.1.4 Sistemas de protección

3.2 El panel de frontal

3.2.1 Controles

3.2.2 Indicadores

3.2.3 Display

3.3 Interfaz de control remoto y programación

3.3.1 Interfaces de comunicación

3.3.2 Protocolos de comunicación

3.3.3 Comandos SCPI

4

Controlador-de-Temperatura

5

Control y Medición

6

Protecciones

7

Alimentación

8

Interfaz de Usuario

9

Diseño Térmico

10

Diseño Mecánico

11

Integración Electromecánica

12

Implementación electrónica

12.1 Esquematicos

12.2 PCBs

13

Implementación mecánica

14

Implementacion de Firmware

14.1 dependencias

-

15

Validación Eléctrica

16

Validación Térmica

17

Validación del Sistema

18

Manual de Operación

19

Mantenimiento

Parte II

Gestión de Versiones del Producto

20

Historial de Versiones

Parte III

Desarrollo de Firmware

21

Desarrollo de firmware

21.1 Resumen

Estructura y organización y resumen del desarrollo:

Sistema de comunicación: Driver Dependencias y obligaciones API Inyección de dependencias (buffers, etc) Communication Task Mantenimiento de interfaz Manejo de errores Recepción y extracción de mensajes Transmisión API Motor SCPI Búsqueda de comandos Parser SCPI Control de errores Armado de Respuestas Escritura de mensajes de respuesta

Panel Frontal: Pantalla API Control API

Control: Salida de potencia Seguridad API Control PID Modelo incremental para DSP Sensores de temperatura Filtros API

22

Sistema de Comunicación

22.1 Arquitectura General

22.1.1 Modelo de Capas y Responsabilidades

El sistema de comunicación se organiza en tres capas funcionales con responsabilidades estrictamente delimitadas y fronteras bien definidas mediante interfaces abstractas. Esta segmentación permite modificar o reemplazar cualquiera de las capas sin impactar a las demás, siempre que se respeten los contratos establecidos.

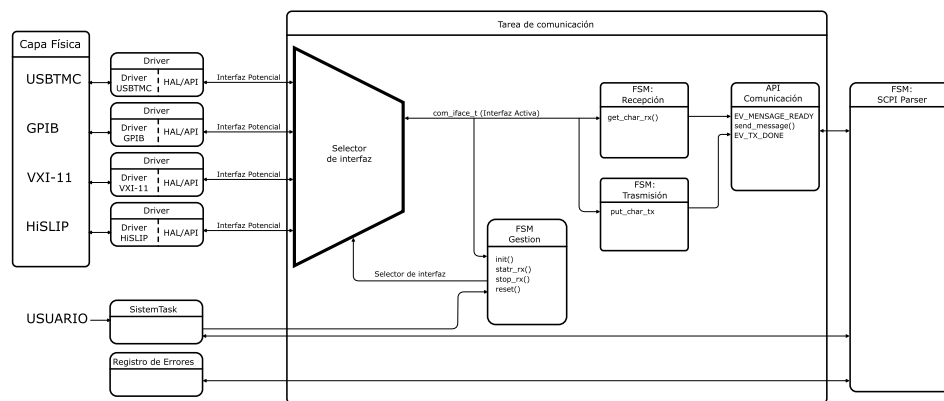


Figura 22.1: Modelo de capas del sistema de comunicación. Las flechas indican las dependencias funcionales y el sentido del flujo de datos.

Capa de Driver (Interfaz de Transporte Físico) Es la capa más cercana al hardware. Su responsabilidad exclusiva es la gestión del periférico de comunicación (UART, USBTMC, SPI, TCP/IP, etc.) y la traducción de eventos físicos a una representación abstracta normalizada.

El driver expone sus servicios a través de la interfaz `comm_iface_t`, que constituye un contrato cerrado. Toda implementación concreta debe cumplir con esta interfaz para ser utilizable por las capas superiores. Esta abstracción oculta por completo las

particularidades del registro de periférico, las interrupciones, el DMA o los protocolos de enlace de bajo nivel.

La capa de driver:

- Administra los recursos de hardware (registros, interrupciones, canales DMA).
- Produce eventos abstractos de interfaz (dato recibido, transmisión completada, error).
- Mantiene el estado operativo de la conexión física.
- No asigna buffers de aplicación ni realiza procesamiento de los datos transportados.

Capa de Middleware (Communication Task) Esta capa constituye el núcleo del sub-sistema de comunicación. Se implementa como una tarea independiente (o un bucle dentro de un esquema cooperativo) y opera exclusivamente sobre la interfaz abstracta proporcionada por el driver.

Sus responsabilidades comprenden:

- La gestión global del ciclo de vida de la interfaz (inicialización, arranque, detención, recuperación ante fallos).
- La recepción de bytes desde el driver y la delimitación de mensajes completos de acuerdo con las reglas del protocolo de instrumentación (detección del terminador LF según IEEE 488.2).
- La transmisión de mensajes hacia el driver cuando la aplicación lo solicita.
- La publicación de eventos lógicos hacia la capa de aplicación: EV_MESSAGE_READY (mensaje disponible) y EV_TX_DONE (transmisión finalizada).
- La coordinación de las máquinas de estado de gestión, recepción y transmisión.

El middleware desconoce por completo el contenido semántico de los mensajes que transporta. Un mensaje es, para esta capa, una secuencia de bytes delimitada por un terminador; no realiza análisis sintáctico ni interpreta comandos.

Capa de Aplicación (Motor SCPI) Es la capa superior, consumidora de los servicios de comunicación. Recibe mensajes completos desde el middleware y los procesa de acuerdo con la sintaxis y semántica del protocolo SCPI (Standard Commands for Programmable Instruments).

Sus responsabilidades son:

- El análisis sintáctico y semántico de los comandos recibidos.
- La ejecución de las acciones solicitadas (consulta de medidas, modificación de parámetros, etc.).
- La generación de respuestas conformes al estándar IEEE 488.2.
- La interacción con el resto del sistema del instrumento (control, medición, configuración).

El motor SCPI no accede directamente al hardware de comunicación ni a los buffers de transporte. Para enviar una respuesta, invoca la función `send_message()` expuesta por el middleware.

Principio de Propiedad de Recursos Cada capa es propietaria exclusiva de los recursos que administra. Este principio evita dependencias implícitas y condiciones de carrera:

- Los buffers de transporte (recepción y transmisión) son propiedad de la *Communication Task* y se inyectan al driver por referencia durante la inicialización. El driver opera sobre ellos pero no los posee.
- El contexto del periférico (registros de configuración, estado interno del driver) es privado de cada implementación concreta del driver y no es accesible desde fuera.
- Los buffers de comandos y respuestas SCPI pertenecen al motor SCPI y son independientes del transporte.

La tabla 22.1 detalla la asignación de propiedad para cada recurso compartido.

Cuadro 22.1: *Propiedad de recursos en el sistema de comunicación.*

Recurso	Propietario	Observación
Buffer RX de transporte	Communication Task	Asignado a la estructura <code>comm_iface_t</code>
Buffer TX de transporte	Communication Task	Asignado a la estructura <code>comm_iface_t</code>
Contexto del periférico	Driver	Privado de cada implementación
Registro de eventos	Driver	Compartido ISR/Task mediante snapshot
Estado de interfaz	Driver	Visible para capas superiores en modo lectura
Buffers de comandos SCPI	Motor SCPI	Independientes del transporte
Buffers de respuesta SCPI	Motor SCPI	Independientes del transporte

Flujo de Información entre Capas La comunicación entre capas sigue un modelo orientado a eventos, sin llamadas bloqueantes. El sentido del flujo es el siguiente:

Recepción: Medio físico → Driver → Eventos de interfaz → Communication Task → Mensaje completo → Motor SCPI.

Transmisión: Motor SCPI → Solicitud de transmisión → Communication Task → Driver → Medio físico.

La capa de aplicación (Motor SCPI) se documenta en detalle en el capítulo siguiente. El resto de este capítulo describe la arquitectura interna de las capas de Driver y Middleware, así como las interfaces que las conectan.

22.1.2 Propiedad de Recursos

La asignación explícita de la propiedad de cada recurso compartido es un principio arquitectónico fundamental del sistema de comunicación. Establecer un propietario

único para cada estructura de datos, buffer o registro de estado persigue tres objetivos:

1. **Eliminar la contención implícita:** cuando dos módulos pueden modificar un mismo recurso sin una mediación clara, aparecen condiciones de carrera y efectos laterales difíciles de depurar.
2. **Permitir la verificación estática:** el propietario es responsable de asignar, liberar y garantizar la validez del recurso; los demás módulos lo utilizan bajo un contrato de acceso definido (lectura, escritura mediada, etc.).
3. **Facilitar la portabilidad:** si cada recurso tiene un dueño bien identificado, reemplazar una capa (por ejemplo, cambiar el driver de USBTMC por uno de TCP/IP) no obliga a modificar la gestión de memoria de las demás capas.

La tabla 22.1 (presentada en la sección anterior) resume la asignación de propiedad para los recursos principales. A continuación se detallan las implicancias de cada caso.

Buffers de Transporte (RX y TX) Los buffers de recepción y transmisión a nivel de transporte son **propiedad exclusiva de la *Communication Task***. Esto significa que:

- La tarea de comunicación reserva estáticamente la memoria necesaria (por ejemplo, en su propio stack o en la sección `.bss` de su módulo objeto).
- Durante la inicialización, las referencias a estos buffers se asignan directamente en los campos `rx_buffer` y `tx_buffer` de la estructura `comm_iface_t` antes de invocar `init()`.
- El driver puede leer y escribir en ellos, pero **no es responsable de su ciclo de vida**. Si un driver necesita dimensionar los buffers, lo hace a través de parámetros de configuración, no asignándolos por sí mismo.
- Esta separación permite que, en un mismo hardware, dos interfaces distintas compartan la misma tarea de comunicación sin que el driver imponga un tamaño fijo de buffer.

Contexto del Periférico Cada implementación concreta del driver mantiene una estructura de contexto privada (registros de configuración, estados internos, handles de sistema operativo, etc.). Este contexto es **propiedad exclusiva del driver** y no es accesible desde la *Communication Task* ni desde la aplicación. Las capas superiores desconocen por completo su contenido y solo interactúan con el driver a través de la interfaz abstracta `comm_iface_t`.

Esta encapsulación garantiza que:

- El middleware no pueda, por error, modificar un registro de periférico sin pasar por la API.
- Se pueda implementar un driver simulado (*mock*) para pruebas unitarias del middleware, reemplazando completamente el contexto real sin que la tarea de comunicación lo advierta.

Registro de Eventos El registro de eventos es una estructura de mapa de bits (*bit-mask*) que el driver actualiza de forma asíncrona (generalmente desde una ISR) y la *Communication Task* lee en cada ciclo. La **propiedad formal** reside en el driver, pero el acceso es compartido:

- Al inicio del siguiente ciclo de la *Communication Task*, se captura atómicamente el registro de eventos del driver en una variable local (*snapshot*) y se limpia el registro original. A partir de este momento, todas las FSM operan sobre la copia local, cada una enmascarando los bits que le corresponden.
- La *Communication Task* lee el registro de manera atómica (mecanismo de *snapshot* descrito en la sección 22.3.2) y luego lo limpia.
- Ninguna otra capa accede directamente a este registro, ni siquiera en modo lectura. La aplicación recibe eventos lógicos de más alto nivel a través de la API de comunicación (EV_MESSAGE_READY, EV_TX_DONE).

Estado de la Interfaz El estado operativo de la interfaz (COMM_STATE_CONNECTED, COMM_STATE_RX_ACTIVE etc.) es mantenido por el driver y **visible en modo lectura** para la *Communication Task*. La tarea de middleware puede consultar este estado para tomar decisiones de control (por ejemplo, no intentar transmitir si la interfaz no está conectada), pero no puede modificarlo directamente. La modificación del estado es consecuencia de las operaciones invocadas a través de la API (start_rx(), stop_rx()) o de eventos de hardware detectados por el driver.

Buffers de Comandos y Respuestas SCPI Los buffers donde se almacenan los comandos recibidos (ya delimitados) y las respuestas generadas por el instrumento son **propiedad exclusiva del Motor SCPI**. Esta decisión es crítica para la independencia de la aplicación respecto al transporte:

- El middleware entrega el mensaje completo a la capa de aplicación copiándolo en el buffer de comandos SCPI o pasando una referencia que la aplicación debe consumir antes del siguiente ciclo de la tarea.
- La aplicación formatea sus respuestas en sus propios buffers y luego invoca send_message() para transferirlas al middleware.
- Si en el futuro se reemplaza el protocolo SCPI por otro (por ejemplo, Modbus), los buffers de la aplicación cambian, pero el middleware de comunicación permanece inalterado.

Principio de Inyección de Dependencias La separación de propiedad se complementa con un patrón de **inyección de dependencias** en tiempo de inicialización. La *Communication Task* no instancia directamente un driver concreto, sino que recibe una referencia a una implementación de comm_iface_t junto con los parámetros de configuración necesarios (incluyendo las referencias a los buffers de transporte). Este meca-

nismo, detallado en la sección 22.2.7, es el que permite que el mismo código de middleware opere sobre UART, USBTMC o TCP/IP sin modificaciones.

22.1.3 Conceptos Fundamentales

Para garantizar una interpretación unívoca de la arquitectura, esta sección define los términos que se emplean de manera recurrente en la documentación del sistema de comunicación. Cada concepto está acotado al contexto de este subsistema; otros módulos del firmware pueden utilizar definiciones complementarias.

Estado Un estado representa una condición operativa persistente de una interfaz de comunicación. Describe una situación que se mantiene durante un intervalo de tiempo y que puede consultarse en cualquier momento.

En el sistema de comunicación, los estados se implementan como una máscara de bits (`comm_iface_state_t`), lo que permite que múltiples condiciones coexistan simultáneamente. Por ejemplo, una interfaz puede estar al mismo tiempo `CONNECTED` y `RX_ACTIVE`.

Los estados son modificados exclusivamente por el driver como resultado de operaciones solicitadas a través de la API (`start_rx()`, `stop_rx()`) o de eventos de hardware detectados internamente. Las capas superiores pueden leerlos, pero no alterarlos directamente.

Evento Un evento es la representación de un suceso instantáneo y asincrónico, producido por el hardware de comunicación o por el propio driver. A diferencia de los estados, un evento no describe una condición permanente sino una notificación puntual que debe ser procesada por las capas superiores.

Los eventos se codifican como una máscara de bits (`comm_iface_event_t`) en la que cada bit representa un tipo de suceso. Esta representación permite:

- La coexistencia de múltiples eventos simultáneos.
- El filtrado eficiente mediante operaciones de enmascaramiento binario.
- La transferencia atómica del registro completo (mecanismo de *snapshot*) entre el contexto de interrupción y el de tarea.

Ejemplos típicos de eventos son la disponibilidad de nuevos datos en el buffer de recepción, la finalización de una transmisión o la detección de una condición de error de hardware.

Error Un error es una condición anómala detectada durante la operación de la interfaz de comunicación. Los errores pueden originarse en el medio físico (errores de paridad, de encuadre, desbordamiento del buffer de hardware), en la capa de transporte (timeout de transmisión, fallo de bus) o en la propia implementación del driver (error interno).

La ocurrencia de un error se notifica mediante los eventos específicos definidos en `comm_iface_event_t` (`IFACE_EVENT_RX_ERROR_OVERFLOW`, `IFACE_EVENT_TX_ERROR_TIMEOUT`, etc.). Adicionalmente, un error crítico puede provocar la activación del estado persistente `COMM_STATE_ERROR`, lo que indica a las capas superiores que la interfaz no se encuentra en condición operativa normal.

Es importante distinguir entre el código de error retornado por algunas funciones de la API (como `init()` o `reset()`) y los eventos de error asincrónicos. Los primeros informan del resultado de una operación solicitada; los segundos notifican condiciones sobrevenidas fuera del flujo de llamadas.

Operación Una operación es una acción explícita solicitada por una capa superior a través de la API de la interfaz de comunicación. Constituye el mecanismo de control de que disponen la *Communication Task* y, de forma mediada, la aplicación, para gobernar el comportamiento del driver sin exponer los detalles de implementación del periférico.

Las operaciones se corresponden con los *callbacks* definidos en la estructura `comm_iface_t`:

- **Ciclo de vida:** `init()`, `deinit()`, `reset()`.
- **Control de flujo:** `start_rx()`, `stop_rx()`, `start_tx()`.
- **Acceso a datos:** `get_char_rx()`, `put_char_tx()`.
- **Gestión de eventos:** `set_event()`, `get_event()`.
- **Protección concurrente:** `protect_rx()`, `unprotect_rx()`, `protect_tx()`, `unprotect_tx()`.

Todas las operaciones están diseñadas para ser no bloqueantes, en coherencia con el modelo de ejecución cooperativa de la *Communication Task*.

Contexto El contexto es una estructura de datos privada asociada a una instancia concreta de interfaz de comunicación. En la implementación, la estructura `comm_iface_t` incluye un campo `void *context` que cada driver utiliza para almacenar la información que necesita: registros de configuración del periférico, handles del sistema operativo, punteros a buffers internos, etc.

Ni la *Communication Task* ni la capa de aplicación acceden al contenido del contexto; solo lo pasan como argumento en cada invocación a los *callbacks* del driver. Esta opacidad garantiza la independencia de las capas superiores respecto de los detalles de implementación del hardware.

Propietario El propietario de un recurso es el componente del sistema responsable de:

- Reservar la memoria o el handle que lo representa.
- Garantizar su validez durante el tiempo de uso.
- Liberarlo cuando ya no es necesario.

La asignación de propiedad está explícitamente documentada en la tabla 22.1. Los componentes que no son propietarios de un recurso pueden utilizarlo únicamente bajo

las condiciones establecidas por el contrato de acceso (por ejemplo, el driver escribe en los buffers de transporte inyectados por la *Communication Task*, pero no es responsable de su asignación ni de su liberación).

Mensaje En el contexto del sistema de comunicación, un mensaje es la unidad de información que se transfiere entre la *Communication Task* y la capa de aplicación. Se define como una secuencia contigua de bytes, delimitada por un carácter de fin de mensaje (el terminador LF, 0x0A, conforme a IEEE 488.2), y despojada de cualquier carácter de control de transporte (como CR, 0x0D).

La delimitación del mensaje es responsabilidad de la máquina de estados de recepción del middleware. Una vez ensamblado el mensaje completo, se notifica a la aplicación mediante el evento lógico EV_MESSAGE_READY. La aplicación puede entonces consumir el mensaje desde el buffer de recepción de la *Communication Task* (o copiarlo a sus buffers propios) y proceder a su análisis sintáctico.

Análogamente, cuando la aplicación desea transmitir información, compone un mensaje en sus buffers de respuesta y solicita el envío invocando `send_message()`, lo que activa la máquina de estados de transmisión del middleware.

Esta definición desacopla completamente el transporte del contenido: el middleware no distingue entre un comando SCPI, una consulta de estado o cualquier otra carga útil; opera exclusivamente con mensajes como secuencias de bytes delimitadas.

22.1.4 Flujo de Comunicación

El modelo de interacción entre capas es puramente asíncrono y orientado a eventos. No existen llamadas bloqueantes ni dependencias cíclicas entre la *Communication Task* y la capa de aplicación. La secuencia de operaciones se describe a continuación para los dos escenarios fundamentales: recepción de un mensaje y transmisión de una respuesta.

Flujo de Recepción

1. El medio físico entrega datos al periférico. La ISR del driver o el DMA transfieren los bytes al buffer de recepción inyectado (buffer circular o estructura de doble buffer, según la estrategia del driver).
2. El driver activa en su registro de eventos el bit correspondiente a `IFACE_EVENT_RX_DATA_AVAILABLE`. Si se detecta una condición de error durante la recepción (overflow, framing, paridad), se activa el bit de error específico en lugar del de datos, o simultáneamente con él.
3. En el siguiente ciclo de la *Communication Task*, se ejecuta la captura atómica del registro de eventos (*snapshot*) y se limpia el registro original.
4. La FSM de Recepción consume los eventos del *snapshot*. Si detecta `IFACE_EVENT_RX_DATA_AVAILABLE` extrae uno a uno los bytes mediante `get_char_rx()` y los acumula en su espacio de trabajo.

5. Cada byte es evaluado contra el terminador LF (0x0A). Los caracteres CR (0x0D) se descartan silenciosamente. Mientras no aparezca LF, la máquina permanece en el estado RX_GATHERING.
6. Al detectar el terminador, la FSM de Recepción considera el mensaje completo, retorna al estado RX_IDLE y publica el evento lógico interno EV_MESSAGE_READY.
7. La capa de aplicación (Motor SCPI), que es notificada de este evento en el mismo ciclo o en el siguiente según la implementación, toma el mensaje desde el buffer de recepción del middleware y procede a su análisis sintáctico.

Flujo de Transmisión

1. La capa de aplicación, una vez generada una respuesta, invoca la función `send_message()` provista por la API de la *Communication Task*, pasando la referencia al buffer de respuesta y su longitud.
2. El middleware copia (o encola) los datos en su buffer de transmisión y activa internamente la señal EV_TX_REQUEST.
3. La FSM de Transmisión, al evaluar este evento, transita al estado TX_BUSY y comienza a transferir bytes al driver mediante llamadas sucesivas a `put_char_tx()`. Una vez cargado el bloque, invoca `start_tx()` para iniciar la transmisión física.
4. El driver maneja el envío (por interrupción o DMA). Al completarse, activa en su registro de eventos el bit `IFACE_EVENT_TX_COMPLETE`.
5. En el siguiente ciclo de la tarea de comunicación, la FSM de Transmisión captura este evento desde el *snapshot*, retorna al estado TX_IDLE y publica el evento lógico EV_TX_DONE hacia la aplicación.
6. Si durante la transmisión se produce un error (timeout, fallo de bus), el driver activa el evento de error correspondiente. La FSM de Transmisión lo procesa, ejecuta `reset()` sobre el canal de transmisión y notifica el error a la aplicación a través de la API de comunicación.

Sincronización y No Bloqueo Todo el ciclo de recepción y transmisión se ejecuta dentro del mismo hilo de la *Communication Task*, sin necesidad de semáforos ni colas adicionales. La aplicación, si se ejecuta en el mismo contexto (esquema cooperativo), procesa los mensajes de forma secuencial después de que la tarea de comunicación complete su ciclo. Si la aplicación reside en una tarea separada, la transferencia se realiza mediante una cola de mensajes —implementada como un buffer circular simple— que garantiza la entrega sin bloqueo.

Este flujo será refinado con las máquinas de estado concretas en la sección de Middleware, pero la descripción aquí presentada constituye el marco de referencia para todo el subsistema.

22.2 Capa de Driver

La capa de driver constituye el punto de contacto entre el *hardware* de comunicación y el *firmware* de nivel superior. Su misión es traducir las particularidades de cada periférico físico a una representación abstracta y homogénea, de modo que el resto del sistema pueda operar sin conocer los detalles de implementación del medio de transporte.

Esta sección describe el contrato que todo driver debe satisfacer —la interfaz abstracta `comm_iface_t`—, los recursos que administra y las estrategias de sincronización que emplea para operar de manera segura en un entorno con concurrencia.

22.2.1 Interfaz Abstracta `comm_iface_t`

La interfaz `comm_iface_t` es una estructura de punteros a función y campos de datos que define el contrato vinculante para cualquier implementación de driver de comunicación. Está diseñada para ser completamente agnóstica respecto del periférico subyacente: UART, USBTMC, TCP/IP o cualquier otro medio pueden ser encapsulados detrás de esta misma interfaz.

La estructura, definida en el código como `comm_iface_t`, contiene los siguientes grupos funcionales:

- **Identificación:** un campo `id` (`comm_iface_id_t`) que identifica unívocamente la interfaz (USART, TCP, USBTMC) y un campo `name` legible para diagnóstico.
- **Contexto privado:** un puntero `opaco void *context` que cada implementación concreta utiliza para almacenar sus datos internos (registros del periférico, handles, buffers auxiliares). Ni la *Communication Task* ni la aplicación acceden a este campo; simplemente lo reenvían en cada invocación a los *callbacks*.
- **Estado y eventos:** los campos `state` (`comm_iface_state_t`) y `event` (`comm_iface_event_t`) mantienen, respectivamente, la condición operativa actual de la interfaz y el registro de eventos pendientes de procesar.
- **Buffers de transporte inyectados:** punteros `rx_buffer` y `tx_buffer` a sendos *buffers circulares*. Estos buffers son propiedad de la *Communication Task* y se asignan a la estructura antes de invocar `init()`. El driver los utiliza como espacio de trabajo para la transferencia de datos, pero no es responsable de su asignación ni de su liberación.
- **Callbacks de ciclo de vida:** `init()`, `deinit()` y `reset()`. Gestionan la inicialización, desinicialización y reinicio del periférico. Retornan un código de error de tipo `comm_error_t` para informar del resultado de la operación.
- **Callbacks de control de flujo:** `start_rx()`, `stop_rx()` y `start_tx()`. Activan y desactivan la recepción, e inician la transmisión de un bloque de datos previamente cargado.
- **Callbacks de acceso a datos:** `get_char_rx()` y `put_char_tx()`. Son las primitivas de lectura y escritura de bytes individuales sobre los buffers de transporte. Ambas

son no bloqueantes: retornan un valor booleano que indica si la operación tuvo éxito.

- **Callbacks de gestión de eventos:** `set_event()` y `get_event()`. Permiten escribir y leer el registro de eventos de la interfaz. `get_event()` es utilizado por la *Communication Task* para obtener el *snapshot* atómico al inicio de cada ciclo.
- **Callbacks de protección concurrente:** `protect_rx()`, `unprotect_rx()`, `protect_tx()` y `unprotect_tx()`. Proveen mecanismos de exclusión mutua para el acceso a los buffers de recepción y transmisión desde contextos concurrentes (ISR y tarea). La implementación concreta de estos *callbacks* depende del hardware y del sistema operativo subyacente; las capas superiores los invocan sin conocer su estrategia interna.

La figura 22.2 muestra un diagrama esquemático de la estructura `comm_iface_t` y su relación con los componentes que la utilizan.

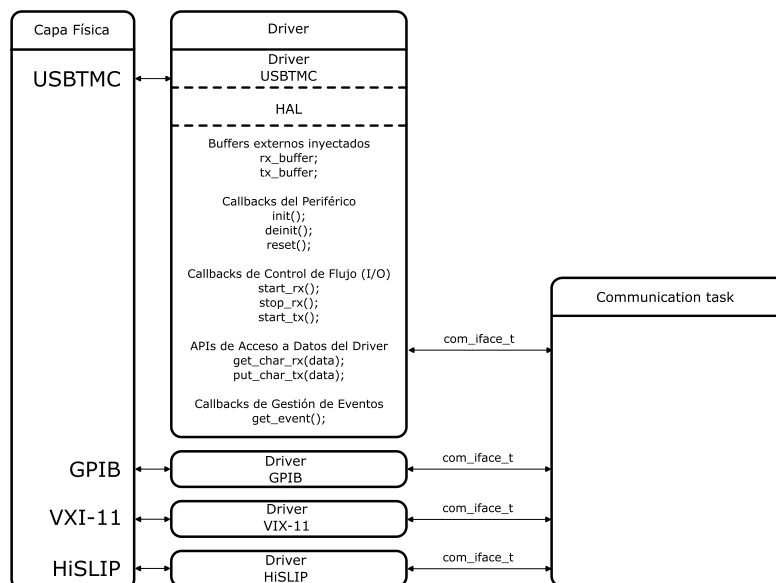


Figura 22.2: Estructura `comm_iface_t` y su interacción con la *Communication Task* y el driver concreto.

La interfaz `comm_iface_t` es el único punto de acoplamiento entre el middleware y el hardware de comunicaciones. Cualquier driver que implemente correctamente estos *callbacks* y respete la semántica de los campos de estado y eventos podrá ser utilizado por la *Communication Task* sin modificar una sola línea de su código.

22.2.2 Estados de la Interfaz**22.2.3 Eventos de Comunicación y Error****22.2.4 APIs de Ciclo de Vida****22.2.5 APIs de Acceso a Datos****22.2.6 Protección de Acceso Concurrente****22.2.7 Estrategias de Sincronización y Gestión de Buffers**

Inyección de Dependencias y Propiedad de Memoria

Buffer Circular Single-Producer-Single-Consumer (SPSC)

Estrategia de Doble Buffer (Even/Odd)

22.3 Capa de Middleware (Communication Task)**22.3.1 Modelo de Ejecución y Planificación****22.3.2 Registro de Eventos y Captura Atómica (Snapshot)****22.3.3 Orden de Precedencia y Flujo de Control****22.3.4 Máquinas de Estado de Transporte**

FSM de Gestión de Interfaz

FSM de Recepción (RX)

FSM de Transmisión (TX)

22.3.5 API de Servicios hacia la Capa de Aplicación**22.3.6 Adaptación a Protocolos de Instrumentación****22.3.7 Selector de Medio de Transporte**

El sistema de comunicación constituye una arquitectura en capas donde cada componente tiene una responsabilidad bien definida y posee explícitamente los recursos que administra.

22.3.8 Propiedad de Recursos

Recurso	Propietario	Observación
Buffer RX Transporte	Communication Task	Inyectado al driver durante el registro de interfaz
Buffer TX Transporte	Communication Task	Inyectado al driver durante el registro de interfaz
Contexto del Periférico	Driver	Privado de cada implementación
Registro de Eventos	Interfaz de Comunicación	Compartido ISR Task mediante Snapshot
Estado de Interfaz	Interfaz de Comunicación	Visible para capas superiores en modo lectura
Buffers de Comandos SCPI	Motor SCPI	Independientes del transporte
Buffers de Respuesta SCPI	Motor SCPI	Independientes del transporte

El driver no realiza asignación dinámica de memoria ni posee buffers propios de aplicación. Los buffers de transporte son propiedad exclusiva de la Communication Task y son inyectados mediante referencias durante la fase de inicialización de la interfaz.

22.4 Interfaz de la Capa Física (Driver)

Cada interfaz física implementa un contrato estricto mediante una instancia de `comm_iface_t`. Esta estructura abstrae completamente las particularidades del hardware subyacente (UART, USBTMC, TCP/IP, etc.) y constituye el único mecanismo de interacción entre la *Communication_Task* y el medio de transporte.

La implementación concreta del driver es responsable exclusivamente de la gestión del transporte físico y de la traducción de eventos de hardware a eventos abstractos de interfaz.

Estados de Interfaz La condición operativa de una interfaz se representa mediante una máscara de estados (*bitmask*):

- `COMM_STATE_CONNECTED`: Existe una conexión física o lógica válida.
- `COMM_STATE_RX_ACTIVE`: La interfaz posee datos pendientes de recepción o se encuentra recibiendo información.
- `COMM_STATE_TX_ACTIVE`: Existe una transmisión física en curso.
- `COMM_STATE_ERROR`: Se detectó una condición de falla persistente en la interfaz.

Los estados representan condiciones persistentes y pueden coexistir simultáneamente.

Eventos de Comunicación Los eventos representan sucesos instantáneos producidos por el hardware o por el propio driver:

- `IFACE_EVENT_INTERFACE_CONNECTED`

- `IFACE_EVENT_INTERFACE_DISCONNECTED`
- `IFACE_EVENT_RX_DATA_AVAILABLE`
- `IFACE_EVENT_TX_COMPLETE`

Eventos de Error Las condiciones anómalas también se notifican mediante eventos:

- `IFACE_EVENT_RX_ERROR_OVERFLOW`
- `IFACE_EVENT_RX_ERROR_FRAMING`
- `IFACE_EVENT_RX_ERROR_PARITY`
- `IFACE_EVENT_TX_ERROR_TIMEOUT`
- `IFACE_EVENT_TX_ERROR_BUS_FAULT`
- `IFACE_EVENT_INTERNAL_ERROR`

La ocurrencia de un evento de error puede provocar adicionalmente la activación del estado persistente `COMM_STATE_ERROR`.

APIs de Ciclo de Vida

- `init()`
- `deinit()`
- `reset()`
- `start_rx()`
- `stop_rx()`
- `start_tx()`

APIs de Acceso a Datos

- `get_char_rx()`
- `put_char_tx()`
- `get_event()`
- `set_event()`

Protección de Acceso Concurrente La interfaz provee mecanismos de protección específicos de cada implementación para sincronizar el acceso concurrente entre ISR y contexto de tarea:

- `protect_rx()`
- `unprotect_rx()`
- `protect_tx()`
- `unprotect_tx()`

Las capas superiores permanecen completamente desacopladas de la estrategia concreta utilizada por cada periférico.

22.4.1 Responsabilidades por Componente

Interfaz de Transporte

La interfaz de comunicación es responsable de:

- Inicializar y configurar la implementación de comunicación asociada.
- Configurar y administrar los recursos necesarios para la transferencia de datos.
- Gestionar la recepción de información proveniente del medio de comunicación.
- Iniciar transmisiones solicitadas por las capas superiores.
- Detectar eventos relevantes producidos por la implementación subyacente.
- Detectar y reportar errores de comunicación.
- Publicar eventos hacia el subsistema de comunicación.
- Mantener el estado operativo interno de la interfaz.

Las siguientes funciones quedan explícitamente fuera del alcance de esta capa:

- Procesamiento de comandos SCPI.
- Interpretación de protocolos de aplicación.
- Validación sintáctica de mensajes.
- Gestión de sesiones.
- Generación de respuestas.
- Almacenamiento persistente de configuraciones.
- Coordinación de tareas de aplicación.

La exclusión de estas responsabilidades permite mantener una separación estricta entre los mecanismos de transporte y la lógica funcional del instrumento.

La Interfaz de Transporte NO posee buffers de aplicación (comandos SCPI, respuestas). Estos pertenecen al Motor SCPI, garantizando que la capa de transporte permanezca independiente del protocolo de aplicación.

22.5 Conceptos Fundamentales

Con el fin de establecer una terminología común para el resto de la arquitectura, la interfaz de comunicación se apoya en los siguientes conceptos fundamentales.

Estado Un estado representa una condición persistente de una interfaz de comunicación durante un intervalo de tiempo.

Los estados describen la situación operativa actual de una interfaz, por ejemplo, si se encuentra disponible, conectada, transmitiendo datos o procesando una recepción.

A diferencia de los eventos, los estados permanecen vigentes hasta que una nueva condición los modifica.

Evento Un evento representa la ocurrencia instantánea de una situación relevante para el sistema.

Ejemplos típicos de eventos son la disponibilidad de nuevos datos, la finalización de una transmisión o la detección de una condición de error.

Los eventos no representan condiciones permanentes sino notificaciones puntuales que deben ser procesadas por las capas superiores.

Error Un error representa una condición anómala detectada durante la operación de la interfaz.

Los errores pueden originarse tanto en el medio de comunicación subyacente como en mecanismos internos de la propia implementación.

La detección de un error no implica necesariamente la interrupción del servicio, pero sí requiere que la condición sea informada mediante los mecanismos definidos por la interfaz.

Operación Una operación representa una acción solicitada por una capa superior sobre una interfaz de comunicación.

Las operaciones constituyen el conjunto de servicios ofrecidos por la interfaz y permiten controlar su comportamiento sin exponer detalles específicos de implementación.

Ejemplos de operaciones son la inicialización de una interfaz, el inicio de una transmisión o el reinicio de una instancia de comunicación.

Contexto El contexto es una estructura de datos privada asociada a una instancia concreta de interfaz.

Su contenido depende de la implementación particular y almacena toda la información necesaria para administrar el medio de comunicación correspondiente.

Las capas superiores no acceden directamente al contexto ni dependen de su estructura interna.

Propietario El propietario de un recurso es el componente responsable de administrar su ciclo de vida y garantizar su validez.

La interfaz define explícitamente la propiedad de los recursos compartidos para evitar dependencias implícitas entre módulos.

Esta definición resulta especialmente importante para las estructuras utilizadas durante la transferencia de datos, permitiendo establecer responsabilidades claras respecto de su creación, utilización y liberación.

22.6 Arquitectura de las Máquinas de Estados (Modelo Mealy)

La *Communication Task* se estructura como un hilo de ejecución independiente (o un bucle operativo controlado en un RTOS) que interactúa con la API del driver mediante

mecanismos no bloqueantes. Para garantizar la modularidad, la portabilidad y la independencia absoluta del hardware, el control lógico se descompone en cuatro Máquinas de Estados Finitos (FSM) independientes basadas en el modelo de Mealy.

En este modelo, las transiciones y las acciones o eventos de salida dependen conjuntamente del estado interno actual y de los eventos de entrada recibidos. Los estados de cada máquina permanecen estrictamente encapsulados y toda intercomunicación se efectúa mediante la evaluación y publicación de eventos de alto nivel. Para optimizar el rendimiento y minimizar la latencia en sistemas embebidos, la señalización de estos eventos se realiza eficientemente mediante mapas de bits (*bitmasks*).

22.6.1 Máquina de Gestión de Interfaz

Esta máquina supervisa el estado global de la interfaz y coordina las acciones de recuperación ante fallas.

Estado Lógico	Evento	Siguiente Estado	Acción
DOWN	CMD_START	READY	init()
READY	CMD_ENABLE_RX	ACTIVE	start_rx()
ACTIVE	CMD_DISABLE_RX	READY	stop_rx()
ACTIVE	Evento de Error Crítico	ERROR	Publicar condición de falla
ERROR	CMD_RESET	READY	deinit(); reset()

Internamente la implementación concreta puede representar estos estados mediante la máscara `COMM_STATE_*`, permitiendo la coexistencia simultánea de múltiples condiciones operativas.

22.6.2 Máquina de Recepción (RX)

La FSM RX consume exclusivamente los eventos publicados por la interfaz de transporte y extrae los bytes mediante `get_char_rx()`.

Estado	Evento	Siguiente Estado	Acción
RX_IDLE	IFACE_EVENT_RX_DATA_AVAILABLE	RX_GATHERING	Leer bytes del driver
RX_GATHERING	Byte distinto de LF	RX_GATHERING	Acumular carácter en buffer
RX_GATHERING	Byte LF	RX_IDLE	Publicar EV_SCPI_MSG_RX
RX_GATHERING	IFACE_EVENT_RX_ERROR_OVERFLOW	RX_IDLE	Publicar error RX
RX_GATHERING	IFACE_EVENT_RX_ERROR_FRAMING	RX_IDLE	Publicar error RX
RX_GATHERING	IFACE_EVENT_RX_ERROR_PARITY	RX_IDLE	Publicar error RX

La FSM RX no interactúa directamente con el hardware ni ejecuta rutinas de recuperación física. La recuperación de la interfaz es responsabilidad exclusiva de la FSM de Gestión.

22.6.3 Máquina de Transmisión (TX)

Esta máquina administra el flujo de salida de datos hacia el medio físico. Responde a las peticiones lógicas del subsistema de procesamiento SCPI y se encarga de transferir los bloques correspondientes al buffer de hardware del driver utilizando sus primitivas de escritura no bloqueantes y ordenando el inicio del envío.

Estado Actual	Evento / Entrada	Siguiente Estado	Acción / Evento Salida
TX_IDLE	EV_SCPI_TX_REQ	TX_BUSY	Cargar datos vía <code>put_char()</code> ; Ejecutar <code>start_transmit</code>
TX_BUSY	EV_TX_COMPLETE	TX_IDLE	Publicar EV_TX_DONE al subsistema SCPI
TX_BUSY	ERR_TX_TIMEOUT	TX_IDLE	Ejecutar <code>reset_transmission()</code> ; Publicar error de TX

22.6.4 Máquina de Procesamiento SCPI

Constituye el núcleo lógico funcional del subsistema de instrumentación. Recibe las tramas asíncronas consolidadas por la máquina de recepción, realiza los análisis sintácticos y semánticos (tokenización), invoca los controladores de ejecución de la aplicación (*handlers*) y despacha las respuestas generadas hacia la máquina de transmisión.

Estado Actual	Evento / Entrada	Siguiente Estado	Acción / Evento Salida
SCPI_IDLE	EV_SCPI_MSG_READY	SCPI_PARSING	Iniciar análisis sintáctico y tokenización de la
SCPI_PARSING	Parser: Error sintaxis	SCPI_IDLE	Encolar código de error SCPI; Publicar EV_SC
SCPI_PARSING	Parser: Comando válido	SCPI_EXECUTING	Invocar <i>handler</i> específico de comando de la a
SCPI_EXECUTING	Handler: Ejecución OK Con respuesta	SCPI_RESPONDING	Preparar buffer de salida; Publicar EV_SCPI_T
SCPI_EXECUTING	Handler: Ejecución OK Sin respuesta	SCPI_IDLE	Liberar recursos de contexto de comandos
SCPI_RESPONDING	EV_TX_DONE	SCPI_IDLE	Limpiar buffers de salida y restablecer contex

22.7 Modelo de Interacción

La comunicación entre la interfaz y las capas superiores se basa en un modelo orientado a eventos. Si tomamos los eventos de interfaz de recepción y los datos de entrada como entradas del sistema, el sistema de recepción será una máquina de Mealy que funcionará con sus estados internos y las entradas mencionadas. En este contexto los eventos pueden ser mucho mejor implementados como un bitmask.

Las capas superiores consumen dichos eventos y toman las acciones correspondientes.

Este enfoque evita dependencias directas entre las implementaciones de comunicación y las tareas de aplicación, favoreciendo una arquitectura desacoplada.

Conceptualmente, la interacción sigue el siguiente flujo:

Medio de Comunicación → Interfaz de Comunicación → Eventos → Subsistema de Comunicación → Motor SCPI

La transmisión de datos sigue el camino inverso:

Motor SCPI → Subsistema de Comunicación → Interfaz de Comunicación → Medio de Comunicación

Un esquema simplificado del sistema se puede ver en la figura 22.3.

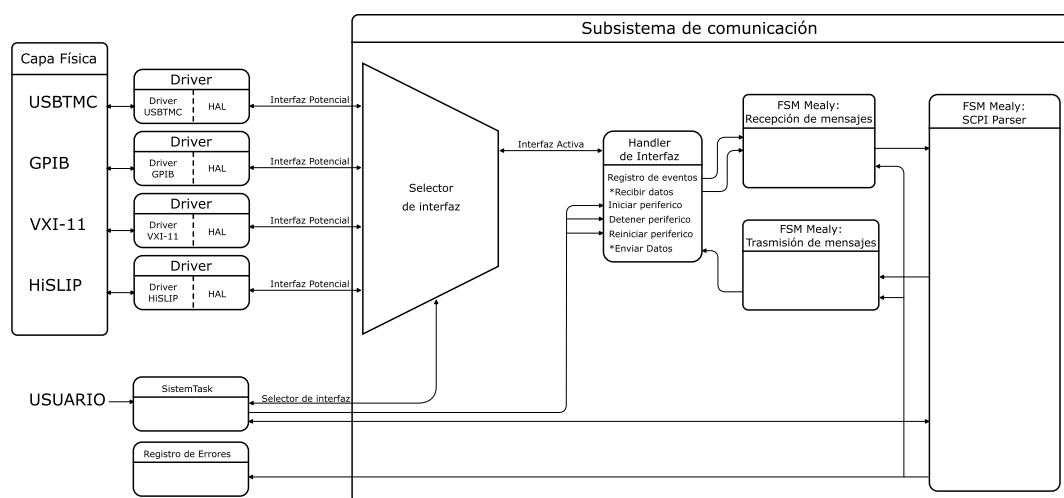


Figura 22.3: Sistema de comunicación con FSMs tipo Mealy.

22.8 Selector de Medio de Transporte

El subsistema de comunicación opera exclusivamente sobre la interfaz abstracta definida por esta capa y no depende de ninguna implementación particular.

La asociación entre el subsistema de comunicación y una implementación concreta es realizada por un selector de interfaz. Este componente determina cuál de las interfaces disponibles será utilizada por el sistema de acuerdo con la configuración activa del instrumento.

El selector de interfaz no participa en la transferencia ni en el procesamiento de datos. Su única responsabilidad consiste en establecer la asociación entre el subsistema de comunicación y una interfaz de comunicación determinada.

Como consecuencia, el medio de transporte puede modificarse sin requerir cambios en las capas superiores del sistema. Un mismo subsistema de comunicación puede operar sobre USBTMC, GPIB, VXI-11, HiSLIP u otras interfaces compatibles manteniendo inalterada la lógica de procesamiento de comandos y respuestas.

22.9 Gestión Interna de Buffers en el Driver

Para ofrecer una API de acceso a datos no bloqueante y eficiente a la *Communication Task*, los controladores de la capa física implementan estrategias de sincronización avanzadas que eliminan la necesidad de exclusión mutua tradicional (primitivas de bloqueo) en entornos de tiempo real (RTOS).

Inyección de Dependencias y Propiedad de Memoria Para garantizar la independencia absoluta del controlador físico (*driver*), la reserva de memoria para los buffers de comunicación (recepción y transmisión) se realiza en el ámbito de la *Communication Task*. El driver no asigna memoria estática ni dinámica por sí mismo.

Durante la fase de inicialización, la tarea inyecta por referencia estos espacios de memoria (punteros y longitudes) al driver mediante la función `init()`. Bajo este esquema:

- La **propiedad de la memoria** (*ownership*) reside en la tarea de comunicación, permitiendo ajustar el tamaño de los buffers según los recursos de la plataforma sin modificar la capa física.
- El **control de acceso** se delega al driver, el cual opera directamente sobre esta memoria inyectada. Las rutinas de servicio de interrupción (ISR) o el hardware subyacente escriben los datos y actualizan los índices de escritura (*head*), manteniendo la estructura conceptual del patrón *lock-free*.

Buffer Circular Single-Producer-Single-Consumer (SPSC) Para periféricos que disponen de canales de acceso directo a memoria (DMA) o soporte nativo por interrupciones capa a capa (como una UART básica):

- La Rutina de Servicio de Interrupción (ISR) o el canal DMA actúa como el **productor** y modifica únicamente el puntero de escritura (*head*).
- La *Communication Task* actúa como el **consumidor** a través de la función `get_char_in()`, modificando únicamente el puntero de lectura (*tail*).
- La detección de desbordamientos (*overflow*) se realiza en el contexto de la ISR comparando la proximidad de ambos punteros, lo que genera de inmediato el evento `ERR_BUFFER_OVERFLOW`.
- La tarea procesa los datos de forma segura hasta la posición del *head* capturada al inicio de su ciclo operativo; las inserciones asíncronas de la ISR no interfieren en la región de memoria leída.

Estrategia de Doble Buffer (Even/Odd) Para periféricos con restricciones severas de hardware o que carecen de soporte DMA generalizado:

- El driver administra dos buffers idénticos en memoria.

- La ISR escribe exclusivamente en el buffer definido como activo. Al completarse una transferencia de bloque por hardware, el driver realiza un intercambio de punteros (*swapping*) de forma instantánea.
- La *Communication Task* consume los datos del buffer que ha pasado a estar inactivo, mientras el hardware continúa llenando de forma aislada el nuevo buffer activo.
- Si al momento de realizar el intercambio por hardware el buffer previo no ha sido procesado por la tarea, el driver bloquea el intercambio y despacha un evento de `ERR_BUFFER_OVERFLOW`.

Ambas estrategias mantienen la responsabilidad de detección de errores de transporte/overflow de buffers en la capa de transporte, preservando la independencia.

22.10 Mecanismo de Despacho de Eventos y Priorización de Ejecución

Para la sincronización entre el controlador físico (*driver*) y la *Communication Task*, se implementa un registro de eventos centralizado en la interfaz abstracta mediante un mapa de bits (*bitmask*). Debido a que múltiples Máquinas de Estados Finitos (FSM) coexisten secuencialmente dentro del mismo hilo de ejecución, el orden de evaluación de este registro determina la prioridad en la resolución de fallos y el procesamiento de datos.

22.10.1 Patrón de Captura Instantánea (*Snapshot*) Atómica

Para evitar condiciones de carrera entre el contexto de las rutinas de servicio de interrupción (ISR) —que escriben de forma asíncrona en el registro— y el contexto de la tarea —que consume los eventos—, la *Communication Task* no accede de forma directa al mapa de bits del driver durante la ejecución de las FSM. En su lugar, el inicio de cada ciclo operativo de la tarea ejecuta de forma obligatoria el siguiente procedimiento:

1. **Captura Atómica:** Mediante una operación de exclusión mutua elemental (como la inhabilitación temporal de interrupciones o primitivas atómicas del procesador), la tarea lee el registro de eventos del driver y lo copia en una variable local denominada *Snapshot de Eventos*.
2. **Limpieza Inmediata:** En la misma sección crítica, el registro de eventos original del driver se borra (o se realiza un *Clear-on-Read*), quedando libre para registrar nuevas notificaciones de la ISR sin interferir con el ciclo actual.
3. **Filtrado por FSM:** Cada máquina de estados extrae exclusivamente los bits de su incumbencia desde el *Snapshot de Eventos* local empleando operaciones de enmascaramiento binario (*AND*).

Este enfoque garantiza la coherencia temporal: todas las FSM ejecutan sus transiciones evaluando exactamente el mismo estado del hardware para ese ciclo de reloj del

sistema.

22.10.2 Orden de Precedencia y Flujo de Control

El procesamiento de los eventos extraídos del *Snapshot* impone una jerarquía estricta. El bucle de la *Communication Task* evalúa las FSM en un orden decreciente de criticidad, asegurando que cualquier anomalía física severa bloquee el procesamiento de datos inválidos en las capas superiores. El orden de ejecución establecido es el siguiente:

1. **FSM de Gestión de Interfaz (Prioridad Máxima):** Evalúa en primera instancia los eventos de error crítico del hardware (ERR_HW_FAIL) y estados de desconexión. Si se detecta un fallo, esta máquina intercepta el control, ejecuta las rutinas de mitigación o reinicio físico del periférico ((reset_reception()/reset_transmission())) y altera el estado global de la interfaz, invalidando los bits de datos del *Snapshot* antes de que pasen a las siguientes etapas.
2. **FSM de Recepción (RX):** Se ejecuta únicamente si la interfaz se encuentra activa y operativa. Extrae los bits correspondientes a la llegada de datos físicos (EV_DATA_RECEIVED) o alertas de saturación menores (ERR_BUFFER_OVERFLOW). Al procesar los caracteres y validar el delimitador LF, genera el evento lógico interno EV_SCPI_MSG_READY.
3. **FSM de Transmisión (TX):** Procesa los eventos relacionados con el vaciado de buffers físicos o la confirmación de envíos (EV_TX_COMPLETE). Su ejecución desacoplada permite liberar recursos de red o puerto serie de forma independiente a la recepción.
4. **FSM de Procesamiento SCPI (Prioridad de Aplicación):** Se ejecuta en la última etapa del ciclo. No consume eventos directos del hardware, sino que reacciona ante los eventos lógicos inyectados por las máquinas de RX y TX en las etapas previas del mismo ciclo, garantizando que el análisis de comandos opere sobre buffers de texto completamente consolidados y validados.

El flujo macroscópico del bucle operativo de la tarea se describe conceptualmente mediante la siguiente secuencia cíclica:

Inicio Bucle → Captura Atómica → FSM Gestión → FSM RX → FSM TX → FSM SCPI → Fin
Bucle / Espera

22.10.3 Adaptación al Protocolo SCPI

Esta sección define cómo la arquitectura segmenta las responsabilidades de procesamiento de datos para la interacción con el motor SCPI.

Detección y Delimitación de Mensajes El controlador físico (*driver*) se limita a extraer bytes del medio y notificar su disponibilidad. La delimitación de los mensajes

es responsabilidad exclusiva de la máquina de estados de recepción (*RX FSM*) de la *Communication Task*. Esta máquina inspecciona secuencialmente los bytes recuperados mediante la API del driver e identifica el fin de mensaje al detectar el carácter LF (0x0A), en conformidad con la norma IEEE 488.2. Los caracteres CR (0x0D) son ignorados de forma silente durante este proceso de ensamblado.

Separación de Preocupaciones

- **Mensaje Ensamblado:** Bloque de bytes consolidado en el buffer interno de la *Communication Task*, cuyo término está marcado por el carácter LF.
- **Comando SCPI:** Estructura jerárquica de nodos analizada y parseada, bajo la propiedad exclusiva del motor SCPI.

La *Communication Task* entrega la cadena de bytes del mensaje completo al motor SCPI una vez validado el delimitador; el motor SCPI asume de forma independiente las tareas de tokenización, análisis sintáctico y ejecución.

Interrupciones de Consulta (Query Interrupts) La capa física no posee noción de interrupciones de consulta. Si el protocolo del periférico subyacente dispone de este mecanismo (por ejemplo, el control de flujo en USBTMC o comandos de desasociación en GPIB), el driver traduce dicha condición de hardware en un evento abstracto de interfaz. La máquina de gestión de la *Communication Task* captura este evento y coordina la acción correspondiente evaluando el estado actual del motor SCPI.

22.11 Entorno de Ejecución y Planificación

La *Communication Task* y sus Máquinas de Estados Finitos (FSM) asociadas están diseñadas para operar de manera determinista dentro de un sistema operativo cooperativo secuencial (arquitectura *bare-metal* o *super-loop*).

Bajo este paradigma, no se requiere la implementación de primitivas de bloqueo complejas (como semáforos o colas de mensajes propias de un RTOS tradicional). Las características del entorno de ejecución son las siguientes:

- **Ejecución No Bloqueante:** Cada FSM evalúa el *Snapshot de Eventos* y retorna inmediatamente el control al planificador si no hay acciones pendientes. El costo computacional (*overhead*) de esta evaluación continua es insignificante.
- **Temporización Simple:** Todo el manejo de tiempos máximos de espera (*timeouts*) para el hardware o la recepción de caracteres incompletos se gestiona mediante una única base de tiempo (*sys_tick*) con resolución de 1 ms.
- **Determinismo:** Al ejecutar todas las fases de comunicación en un único contexto de forma secuencial, se eliminan por diseño las condiciones de carrera y la necesidad de mecanismos de exclusión mutua (*mutexes*) fuera de la captura inicial del *Snapshot*.

22.12 Integración del Parser SCPI (`libscpi`)

Para el procesamiento sintáctico y semántico de los comandos, la arquitectura contempla la delegación de esta responsabilidad a un motor especializado y conforme al estándar IEEE 488.2, como la biblioteca `libscpi`. Esta abstracción simplifica la FSM de Procesamiento SCPI, transformándola en una capa de adaptación (*wrapper*) entre los buffers de transporte y el motor lógico.

La integración bidireccional se establece de la siguiente manera:

Flujo de Recepción (RX → SCPI) Cuando la FSM de Recepción valida la llegada del carácter delimitador (LF) y publica el evento `EV_SCPI_MSG_READY`, la tarea inyecta el buffer continuo de texto directamente a la biblioteca mediante su API de entrada (ej. `SCPI_Input()`). La biblioteca asume internamente el recorrido del árbol de comandos y la ejecución secuencial de los *callbacks* asociados a cada instrumento.

Flujo de Transmisión (SCPI → TX) Las respuestas generadas por los *callbacks* de la aplicación no acceden directamente al hardware. Se enruta la función de escritura de la biblioteca (ej. el *callback* `SCPI_Write()`) para que inserte los datos en el buffer de salida de la tarea y levante el evento lógico `EV_SCPI_TX_REQ`. En el siguiente ciclo del planificador, la FSM de Transmisión detectará este evento y orquestrará el vaciado del buffer hacia el medio físico sin bloquear la ejecución del motor lógico.

Escalabilidad de Contexto Aunque el diseño actual ejecuta el motor SCPI de manera cooperativa dentro del mismo bucle de comunicaciones, esta delimitación estricta de interfaces mediante buffers y eventos permite, en futuras iteraciones del proyecto, migrar el procesamiento SCPI a una tarea de menor prioridad independiente, requiriendo únicamente la adición de una cola (*queue*) entre ambas capas lógicas.

23

Motor SCPI

[Motor SCPI]

Este módulo consume la API de mensajes del Sistema de Comunicación...

23.1 Integración del parser libscpi

23.2 Flujo de recepción y análisis de comandos

23.3 Generación y transmisión de respuestas

Anexos

